

Vulnerability Analysis

The provided code suffers from several severe security flaws:

1. **Plaintext Password Storage:** Passwords are stored directly as strings without hashing. If the memory or data store is compromised, all user credentials are leaked.

Python

```
self.password = password
```

2. **SQL Injection/Logic Flaw:** The authentication mechanism simulates a classic SQL injection via string manipulation logic. Passing *"admin' OR '1'='1"* bypasses validation because it mimics database logic flaws.

Python

```
if user.username == username and user.password == password:
```

3. **Weak Password Policy:** The system accepts trivial passwords like *"admin123"* and *"password"* without checking complexity, length, or entropy.

Python

```
auth_system.add_user("admin", "admin123")
```

4. **No Input Validation:** The system blindly trusts input fields, accepting arbitrary special characters, malformed lengths, or malicious attack vectors.
5. **Data Exposure and Lack of Encapsulation:** In the original code, the User class attributes are entirely public.

Python

```
class User:  
  
    def __init__(self, username, password):  
  
        self.username = username  
  
        self.password = password
```

Because Python doesn't enforce strict access modifiers, any outside code can bypass the AuthenticationSystem entirely, loop through `auth_system.users`, and directly read or modify the password property of any object.

6. **Side-Channel Timing Attacks:** The string comparison used in the loop is vulnerable to a timing attack:

Python

```
if user.username == username and user.password == password:
```

Standard string evaluation (`==`) in Python optimizes performance by checking characters one by one from left to right. The moment it hits a character that

doesn't match, it stops and returns False. An attacker can measure the microscopic differences in response times to determine exactly how many characters of a username or password they got right, allowing them to guess credentials character by character.

7. **No Rate Limiting:** The code loops indefinitely across authentication attempts without a cool-down or tracking mechanism, exposing it directly to brute-force automated attacks.

Refactored Secure Implementation

The refactored code below addresses all the identified vulnerabilities by integrating bcrypt for password hashing, implementing a strong regex-based password policy, sanitizing inputs, and incorporating a tracking dictionary for simple IP/User-based rate limiting.

Python

```
import re
```

```
import time
```

```
import bcrypt
```

```
class User:
```

```
    def __init__(self, username, password):
```

```
        # Input Validation & Sanitization
```

```
        self.username = self._sanitize_username(username)
```

```
        self.password_hash = self._hash_password(password)
```

```
    def _sanitize_username(self, username):
```

```
        if not username or not re.match(r"^[a-zA-Z0-9_-]{3,20}$", username):
```

```
            raise ValueError("Username must be alphanumeric and between 3-20  
characters.")
```

```
        return username.strip()
```

```
    def _hash_password(self, password):
```

```
        # Enforce Password Policy
```

```
        if len(password) < 8 or not any(c.isdigit() for c in password) or not  
any(c.isalpha() for c in password):  
  
            raise ValueError("Password must be at least 8 characters long and contain  
both letters and numbers.")
```

```
# Secure Password Hashing
```

```
salt = bcrypt.gensalt()
```

```
return bcrypt.hashpw(password.encode('utf-8'), salt)
```

```
def verify_password(self, password):
```

```
# Constant-time comparison to prevent timing attacks
```

```
return bcrypt.checkpw(password.encode('utf-8'), self.password_hash)
```

```
class AuthenticationSystem:
```

```
    def __init__(self):
```

```
        self.users = {}
```

```
        self.failed_attempts = {} # Format: {username: [timestamps]}
```

```
    def add_user(self, username, password):
```

try:

new_user = User(username, password)

if new_user.username in self.users:

return "User already exists."

self.users[new_user.username] = new_user

return "User added successfully."

except ValueError as e:

return f"Registration failed: {e}"

def authenticate(self, username, password):

Input Sanitization prevents the logic/injection attack entirely

if not username or not isinstance(username, str):

return False

Simple Rate Limiting (Max 3 attempts per minute)

now = time.time()

attempts = self.failed_attempts.get(username, [])

attempts = [t for t in attempts if now - t < 60]

self.failed_attempts[username] = attempts

```
if len(attempts) >= 3:
```

```
    print(f"Account locked temporarily for user: {username}. Too many requests.")
```

```
    return False
```

```
# Safe lookup removes vulnerability to malformed string logic
```

```
user = self.users.get(username)
```

```
if user and user.verify_password(password):
```

```
    self.failed_attempts[username] = [] # Reset on success
```

```
    return True
```

```
# Track failed attempts
```

```
self.failed_attempts.setdefault(username, []).append(now)
```

```
return False
```

```
# Usage
```

```
auth_system = AuthenticationSystem()
```

```
# Test Secure Registration
```

```
print(auth_system.add_user("admin", "admin123")) # Fails policy
```

```
print(auth_system.add_user("admin_user", "SecurePass123!")) # Success
```

```
# Test Injection Attempt
```

```
malicious_input = "admin' OR '1'='1"
```

```
print(auth_system.authenticate(malicious_input, "anything")) # Output: False (Safe)
```